Next.js

Evolução, App Router e boas práticas modernas

1. Introdução e evolução do Next.js

O que mudou de verdade?

- **Pages Router**: o Next.js surgiu como uma solução para React com SSR, SSG e roteamento por arquivos.
- **App Router**: a grande mudança veio com o Next.js 13, introduzindo a pasta `app/` e o modelo de **React Server Components**.
- **Next.js 15 e 16**: o App Router virou o padrão recomendado, com foco em performance, cache inteligente, Turbopack, Server Actions e experiência de desenvolvimento mais moderna.

Características atuais do Next.js

- **Roteamento baseado em arquivos** com `app/`: a estrutura da pasta define automaticamente as rotas da aplicação. Isso deixa o desenvolvimento mais organizado e reduz a necessidade de configurar roteadores manualmente.
- **Server Components por padrão**: os componentes podem renderizar no servidor, buscar dados diretamente e reduzir a quantidade de JavaScript enviado ao navegador. Isso melhora performance e simplifica o fluxo de dados.

- **Streaming e carregamento incremental**: o Next.js pode entregar partes da página conforme elas ficam prontas, permitindo que o usuário veja conteúdo mais rapidamente, sem esperar todo o carregamento da rota.
- **Cache mais previsível e controlado**: o framework oferece estratégias de cache por requisição, por revalidação e por tags, o que ajuda a equilibrar velocidade e atualização de dados em aplicações reais.
- **Metadata e SEO otimizados**: com `metadata`, é possível definir títulos, descrições e Open Graph de forma nativa, melhorando a indexação e a experiência de compartilhamento em redes sociais.
- **Melhor DX com Turbopack e DX tooling**: ferramentas modernas como o Turbopack aceleram o desenvolvimento ao melhorar a inicialização, rebuilds e experiência de codificação em projetos reais.

2. O App Router

O App Router é a forma moderna de estruturar uma aplicação Next.js. Ele substitui o modelo antigo de `pages/` e traz um sistema mais flexível para layouts, rotas, componentes e dados.

Estrutura básica

```
app/
├── layout.tsx
├── page.tsx
├── globals.css
├── loading.tsx
├── error.tsx
├── not-found.tsx
├── dashboard/
│   ├── layout.tsx
│   └── page.tsx
└── blog/
    └── [slug]/
        └── page.tsx
```

Arquivos especiais

- `layout.tsx`: layout compartilhado para um segmento da rota
- `page.tsx`: conteúdo da página
- `loading.tsx`: estado de carregamento
- `error.tsx`: tratamento de erro da rota
- `not-found.tsx`: página 404
- `template.tsx`: alternativa ao layout para re-montagem

Navegação com `Link`

```
import Link from 'next/link';

export default function HomePage() {
  return (
    <nav>
      <Link href="/dashboard">Dashboard</Link>
      <Link href="/blog/nextjs">Blog</Link>
    </nav>
  );
}
```

Rotas dinâmicas

```tsx
// app/blog/[slug]/page.tsx

type Props = {
  params: Promise<{ slug: string }>;
};

export default async function BlogPostPage({ params }: Props) {
  const { slug } = await params;
  return <h1>Post: {slug}</h1>;
}
```

> Em versões mais recentes, `params` pode ser tratado como `Promise` em rotas dinâmicas.

3. Server Components e Client Components

Por padrão, o Next.js usa Server Components

Isso significa que o componente roda no servidor e pode:

- buscar dados diretamente
- acessar recursos de backend
- reduzir o bundle do cliente
- melhorar performance

Quando usar `use client`?

Quando precisamos de:

- eventos do navegador (`onClick`, `onChange`)
- estado local com `useState`
- hooks como `useEffect`
- APIs do navegador

```
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return <button onClick={() => setCount(count + 1)}>Contador: {count}</button>;
}
```

4. Data fetching moderno

`fetch` no App Router

O Next.js estende a API nativa de `fetch`, permitindo controle de cache e revalidação de dados.

```
async function getPosts() {
  const res = await fetch('https://api.example.com/posts', {
    cache: 'no-store',
  });

  if (!res.ok) throw new Error('Erro ao buscar posts');
  return res.json();
}

export default async function PostsPage() {
  const posts = await getPosts();

  return (
    <ul>
      {posts.map((post: any) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Cache e revalidação

```
fetch('https://api.example.com/products', {
  next: { revalidate: 60 },
});
```

- `cache: 'no-store'`: dados dinâmicos a cada requisição
- `next: { revalidate: 60 }`: revalida a cada 60 segundos
- `cache: 'force-cache'`: usa cache estático quando possível
- `next: { tags: ['products'] }`: cache por tag para invalidação seletiva

Revalidação por tag

```
import { revalidateTag } from 'next/cache';

export async function revalidateProducts() {
  revalidateTag('products');
}
```

Esse padrão é muito usado em sistemas com conteúdo que precisa ser atualizado após ações do usuário.

5. Route Handlers

Route Handlers são usados para criar endpoints de API dentro do App Router.

```
// app/api/hello/route.ts
import { NextResponse } from 'next/server';

export async function GET() {
  return NextResponse.json({ message: 'Olá do Next.js moderno!' });
}

export async function POST(request: Request) {
  const body = await request.json();
  return NextResponse.json({ received: body });
}
```

`/api/hello` pode ser consumido por frontend, app mobile ou webhook externo.

6. Server Actions

Server Actions são a maneira moderna de lidar com mutações de dados, sem criar endpoint manualmente para cada operação.

Como funcionam?

1. Criar uma função assíncrona com `'use server'`
2. Usá-la diretamente em `form` ou evento do cliente
3. O código roda no servidor de forma segura

Exemplo prático

```
'use server';

import { revalidatePath } from 'next/cache';

const todos = ['Aprender Next.js'];

export async function addTodo(formData: FormData) {
  const todo = formData.get('todo') as string;
  if (!todo) return;

  todos.push(todo);
  revalidatePath('/todos');
}
```

```
import { addTodo } from './actions';

export default function TodosPage() {
  return (
    <form action={addTodo}>
      <input type="text" name="todo" />
      <button type="submit">Adicionar</button>
    </form>
  );
}
```

Vantagens

- elimina boilerplate de API
- simplifica operações de CRUD
- integra bem com formulários HTML
- reduz a necessidade de um client-side state complexo para ações simples

7. SEO e metadata

O Next.js moderno oferece suporte nativo para metadados e SEO.

```
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Meu Blog',
  description: 'Artigos sobre desenvolvimento web',
};

export default function Page() {
  return <h1>Bem-vindo</h1>;
}
```

Também é possível criar metadados dinâmicos por rota, além de Open Graph e tags de redes sociais.

8. Boas práticas no Next.js atual

- Prefira o **App Router** em projetos novos
- Use **Server Components** para dados e renderização no servidor
- Use **Client Components** apenas quando necessário
- Evite fetch duplicado e cache desnecessário
- Entenda o comportamento do cache antes de otimizar
- Use `loading.tsx`, `error.tsx` e `not-found.tsx` para melhorar UX
- Mantenha ações e dados no servidor quando possível

9. Conclusão

Resumo

- O **App Router** é o padrão atual do Next.js
- **Server Components** são a base da arquitetura moderna
- **fetch + cache + revalidate** controlam a estratégia de dados
- **Route Handlers** resolvem APIs internas
- **Server Actions** simplificam mutações e formulários
- **SEO e performance** ficam mais fortes com a abordagem moderna

Pratique!

A melhor forma de aprender Next.js é construir projetos reais:

- blog
- dashboard
- ecommerce
- painel administrativo
- app com autenticação e CRUD

Obrigado!